Project 4: Auto Loot

11/5/2025

50 Points Possible

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

Details

Introduction

You are still working for the MMORPG company, and things couldn't be better. You've gotten promoted again due to your work countering the griefers, and now you're spending a lot more time working on gameplay.

Recently, you've been talking with the designers about ways to streamline the game. In particular, you're looking for ways to let players shut off their brains as the game plays itself. To this end, you come up with an amazing idea involving loot.

Loot is very streamlined in your game. Whenever the player defeats a dungeon, all the loot they picked up is stored in a giant loot pile at the beginning. There, the player can choose what they want to take back to town with them. As long as what they carry is below their carrying weight, they can leave with it. However, players cannot leave with more than they can carry.

Looking through a giant pile loot is fun, but the power-gamers find it kind of tedious. They just want to get the loot and go sell it to get to the next dungeon. This is what gave you your idea: Auto loot!

The idea is simple: add a button that will automatically pick the best loot that the player can carry. This will streamline the dungeon crawling experience substantially.

You start hacking away and quickly realize that the naive recursive algorithm won't work. In order to be useful, the algorithm needs to take fewer than 5 seconds on a full inventory of 128 items, and the naive algorithm is timing out after more than a minute. Luckily for you, you were a great student and payed close attention in your Design of Algorithms class back when you were in college. You quickly recognize the looting problem as an example of the "0-1 knapsack" problem.

The knapsack problem is a famous problem. It works like this. Suppose you have a backpack/knapsack that can carry 50 pounds, and there are valuable items all around. You have to choose the most valuable items that will fit in your 50 pound backpack, such that you have chosen the best possible collection of items. Of course, the backpack's weight, the items available to you, their weights and values, all of these are parameters that can change.

At first glance, it seems like an exponential time problem. The naive algorithm is to try every combination of items. The problem is that the player's inventory can have up to 128 unique items in it, leading to 2^128 combinations.

However, using a clever dynamic programming algorithm, it's possible to solve this problem in pseudo-polynomial time. Meaning, O(n*w), where n is the number of items, and w is the carry weight.

After some googling, you find a great treatment of the problem along with some psuedo-code on wikipedia: https://en.wikipedia.org/wiki/Knapsack_problem

Your task is this: given a carry weight and a list of items, their values, and their weights, print the items that the player should bring to maximize their loot value. You should also print the total weight carried, the total value carried, and the amount of time in microseconds the computation took.

Warning: a lot of people who play RPGs like to use "value/weight" as a metric to help them decide what items to loot. You might think that you can sort items by value/weight and keep taking items until your backpack fills up. This is a pretty good algorithm, but it's not *optimal*. See sample_input3.txt for an example of where this algorithm would not give the best result.

Input / output

All input comes from standard input. All output must go to standard output. Do not open a file.

Input data starts with an integer that represents the carry weight. Each line afterward contains 3 pieces of information: an item name (string), a weight (integer), and a value (integer). These fields are separated by semicolons (';') with no space in between.

```
120
rusty mail leggings;10;2
electric piano;110;1500
battery-powered skateboard;4;100
boxing gloves;1;50
diamond-studded boxing gloves;1;2500
grand piano;120;2499
```

The output must be a list of items in the same format and in the same order that they appear in the input, except with a comma and space as separator instead of semi-colon (to make it easier to read). After the last item, print the total weight and value. Match the output *exactly*.

```
electric piano, 110, 1500
battery-powered skateboard, 4, 100
boxing gloves, 1, 50
diamond-studded boxing gloves, 1, 2500
final weight: 116
final value: 4150
```

For C programmers: the maximum weight you need to handle is 1200. The maximum number of items is 128. The maximum length of an item name is 127 characters.

All the inputs and expected answers are attached to this module.

Grading

25% Effort points
75% Correct output and reasonable runtime on all attached outputs (15% pts each)

All your code, including loading the files, must run within 5 seconds to receive anything other than discretionary credit. Otherwise it will be a timeout, the same as if output was incorrect.

Your output should match the correct format exactly, with the correct capitalization, spacing, and punctuation. The TA may be using a diff tool to check your output, so any differences will make things hard for them.

The TA will enable optimizations if they are available in your language.

All tested inputs and outputs are attached.

Programming Languages

You must use C. This applies to every project in this class.

Your code will be compiled with -std=c11 -O2 -lm

How to submit

Submit your code as a single C file. If you used multiple C files or a makefile, I'm sorry to make you do this, but please find a way to cram everything into one .c file (Canvas is not really designed for programming projects, and it becomes problematic to work with if we use .zip files).

Note: you must submit on AutoLab if it is available. We will announce whether you have to submit on AutoLab and where. (https://autolab.encs.vancouver.wsu.edu/courses/CS-450-F24/assessments/Project1)

Name your main file main.c.

Failure to submit correctly could result in a small penalty (5 to 10 points).

Rules

- Your code must run within 5 seconds to avoid timing out. This is unlikely to be a problem. However, if your code runs much more slowly than your classmates who are using the same language, that might be a sign that you are doing something incorrect.
- You are allowed to use standard libraries, however you must write the algorithm yourself.
- If your code submission is corrupt, it will be the same as if you did not submit anything at all. Check your submissions afterward by downloading your own submission and running it.

Plagiarism

No plagiarism. It's okay to read about techniques online, but your code must be your own work. You may not work in groups. You may discuss the problem with classmates, but may not share code. Your code may be checked using an automated plagiarism checker. This is an individual assignment.

Tips

- Use a diff program (WinMerge is one on windows) to compare your output with the sample output and make sure they are identical.
- This problem can be hard to debug. There is a bottom-up method listed in the wikipedia article that is expected to be slower but easier to debug. You can look at all the values in the table to find the first one that is wrong.
- As usual, make sure you valgrind and debug it!

Alignments

- MO1: Entire assignment
- MO3: Correct output and runtime with choice of algorithm (bottom-up or top-down) (75%)

View Rubric

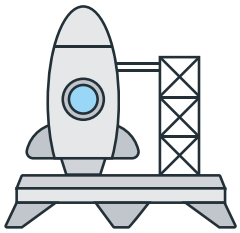
Project 4 rubric

Criteria	Ratings					Points
MO1 **MO1** view longer description Threshold: 4 pts	**Exceeds Mastery** **5 pts**	**Mastery** **4 pts**	**Near Mastery** **3 pts**	**Below Mastery** **2 pts**	**No Evidence** **1 pts**	--/5 pts
MO3 **MO3** view longer description Threshold: 4 pts	**Exceeds Mastery** **5 pts**	**Mastery** **4 pts**	**Near Mastery** **3 pts**	**Below Mastery** **2 pts**	**No Evidence** **1 pts**	--/5 pts

Choose a submission type

Upload

Office 365



Choose a file to upload

File permitted: C

or

Canvas Files

I agree to the tool's **End-User License Agreement**
(https://washingtonstateuniversitysystem.turnitin.com/integrations/cpf/eula/washingtonstateuniversitysystem)
This assignment submission is my own, original work
*

(https://wsu.instructure.com/courses/1820402/modules/items/24229233)

Assignment

(https://wsu.instructure.com/courses/1820402/modules/items/242292